



**UNIVERSITY OF RWANDA
CST-RMA
COMPUTER ENGINEERING
LEVEL 3**

ON MARCH 29, 2026

REG NO: 223027475

FLUTTER LAB 5: POSTS MANAGER WITH SQLITE

Introduction

This lab focuses on building a Flutter mobile application that works **without internet connection** by storing data locally using SQLite.

The goal is to understand how Flutter applications manage local databases, perform CRUD operations, and update the UI dynamically.

The application developed is called “**Offline Posts Manager**”, which allows users to manage posts stored directly on the device.

Application Description

The mobile application enables staff members to:

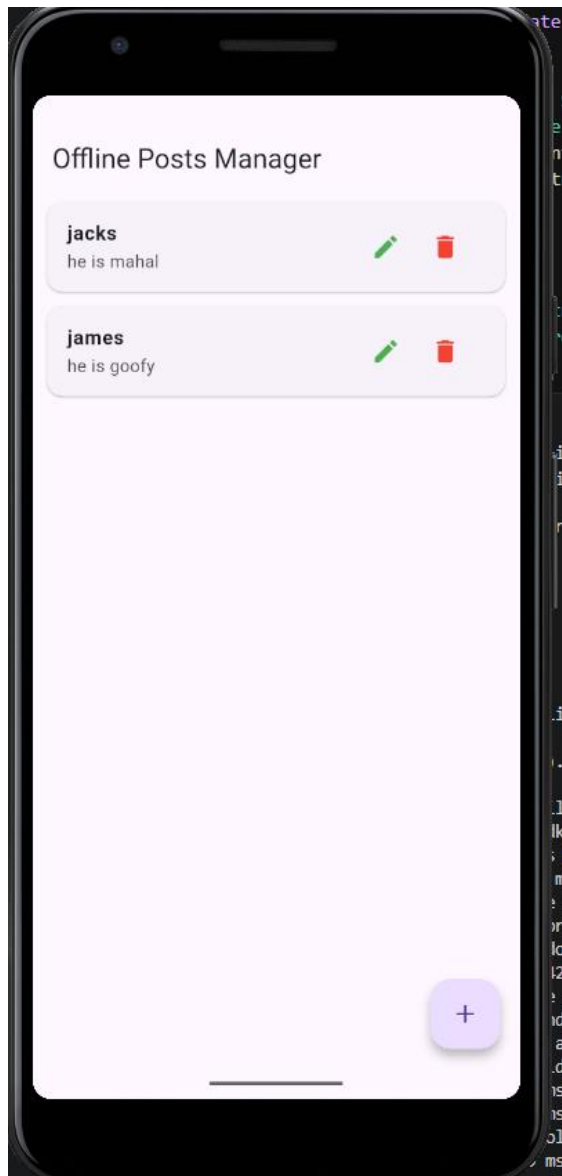
- View all posts stored locally
- Read the details of a post

- Add a new post
 - Edit an existing post
 - Delete a post
-

Application Screens (UI Design)

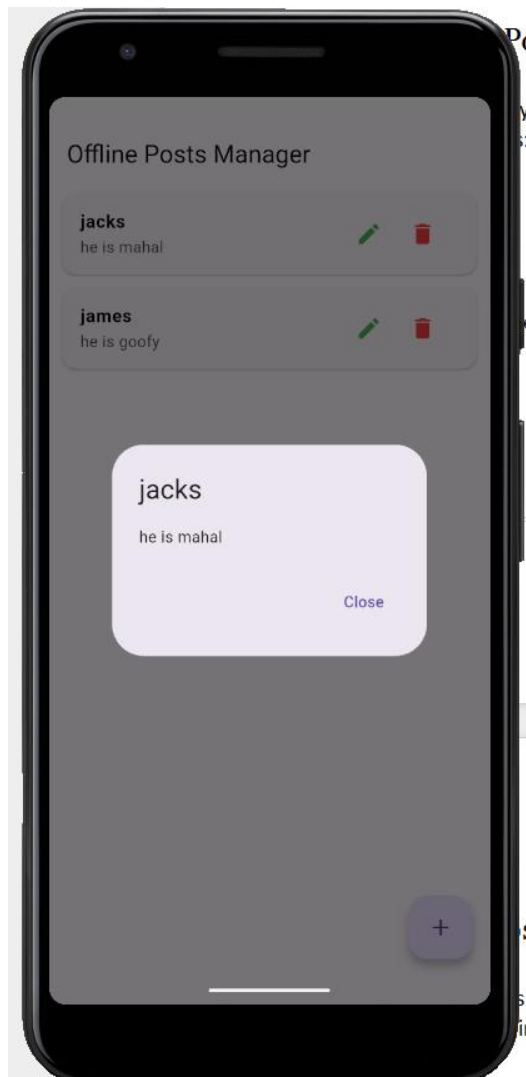
1. Home Screen (Posts List)

- Displays all posts stored in the SQLite database
- Uses a `ListView` to show posts
- Includes a **Floating Action Button (+)** to add a new post
- Each post has options to:
 - View details
 - Edit
 - Delete



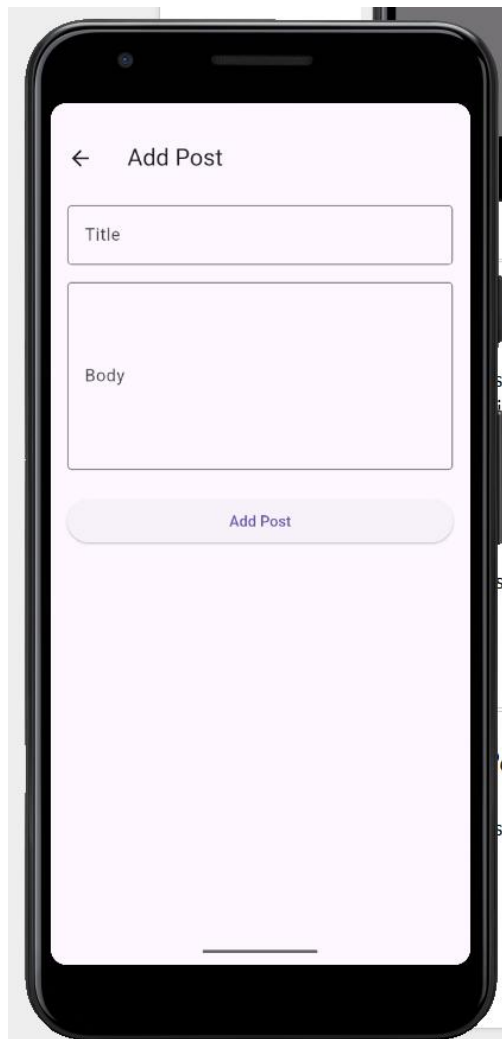
2. View Post Screen (Details Screen)

- Displays full details of a selected post
- Shows:
 - Post ID
 - Title
 - Body
- Provides a clean and readable layout



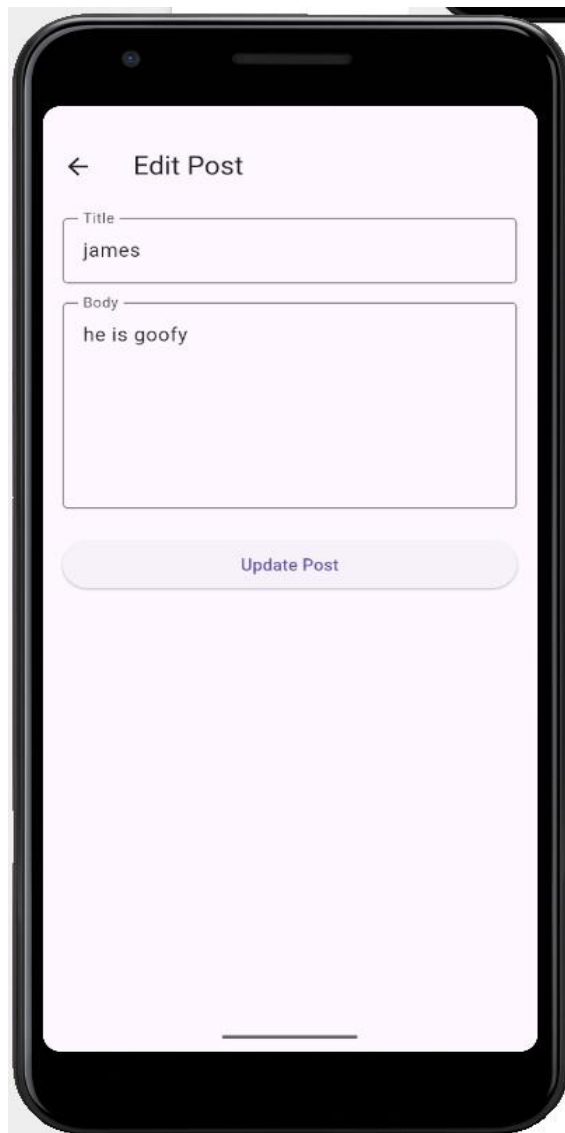
3. Add Post Screen

- Allows users to create a new post
- Contains:
 - Title input field
 - Body input field
 - Save button
- Inserts data into SQLite database



4. Edit Post Screen

- Allows users to update an existing post
- Fields are pre-filled with existing data
- Updates data in the database



5. Delete Post Feature

- Users can delete a post
- Removes the post from SQLite database
- UI updates immediately after deletion

1. Dependencies Used and Why

sqflite package

- Used to interact with SQLite database in Flutter
- Provides functions to:
 - Create database

- Insert data
- Query data
- Update data
- Delete data

path package

- Used to get the correct file path for storing the database
- Ensures the database is saved in the device storage

Why SQLite is necessary

SQLite allows the application to:

- Store data locally on the device
- Work without internet connection
- Persist data even after the app is closed

223027475

LAB5

Q1. Dependencies used

- Sqflite
 - ⊕ used to create and manage a local SQLite database
 - ⊕ Allows performing CRUD operations
- path-provider
 - ⊕ used to locate the correct directory on the device to store the database file
 - ⊕ Ensure the SQLite database is stored safely and can be accessed by the app.
- built-in flutter / dart libraries
 - flutter CSAT: for building the app interface
 - dart: async: for handling asynchronous operations
 - dart: io: for working with files and directories if needed

Why SQLite is necessary to store data locally?
because:

- Offline access: users can view, add, edit or delete posts even without an internet connection.
- Persistent storage: Data remains on the device even after the app is closed
- Structured data: SQLite stores complex, relational data (like posts with title, body and id) efficiently.
- Reliability: SQLite is a proven, lightweight database that integrates easily with flutter.

② To Handle database exceptions

① Database not initialized

Before performing any operation, always check if the database is open and initialized

```
final db = await DatabaseHelper.instance.database;  
if (db == null) {  
    throw Exception("Database not initialized")  
}
```

- If db isn't created yet, any CRUD function will fail. This will ensure we only operate after the db is ready

2. Handling Database Exceptions

Handling database errors is important to prevent crashes and data loss.

1. Database Not Initialized

- Occurs when database is used before opening
- Solution:
 - Ensure database is initialized before operations

2. Insert / Update / Delete Errors

- Occur when operations fail
- Handled using try-catch blocks

Example:

- Insert fails due to invalid data
- Update fails due to missing ID

3. Invalid or Corrupted Data

- Occurs when stored data is incorrect
- Handled by:
 - Validating input before saving
 - Using proper data types

(11). Invalid or corrupted data

Validate data before inserting or updating, and check query results when reading.

```
if (post.title.isEmpty || post.body.isEmpty) {  
    throw Exception("Invalid post data");  
}
```

```
final List<Map<String, dynamic>> results =  
    await db.query('posts');  
if (results.isEmpty) {  
    print("No post found");  
}
```

- Ensure only valid data is stored, and reading operations handle empty or invalid results safely.

⑪ Insert/Update/Delete errors.

wrap all CRUD operations in try-catch blocks to catch runtime errors.

```
try {  
  await db.insert('posts', post.toMap());  
} catch (e) {  
  print("Insert error: $e");  
}  
  
try {  
  await db.update('posts', post.toMap(),  
    where: 'id=?', whereArgs: [post.id]);  
} catch (e) {  
  print("Update error: $e");  
}  
  
try {  
  await db.delete('posts', where: 'id=?', whereArgs:  
    [post.id]);  
} catch (e) {  
  print("Delete error: $e");  
}
```

- Any CRUD error will be caught and logged instead of crashing the app.

3. SQLite in Flutter

Database vs Table

- **Database** → A container that holds data
Example: posts.db

- **Table** → A structure inside the database
Example: `posts` table
-

How Flutter interacts with SQLite asynchronously

Database operations take time, so Flutter uses:

- `Future`
- `async/await`

Example:

- Fetching posts returns a `Future<List<Post>>`
- UI updates after data is loaded

This ensures:

- Smooth performance
 - No UI freezing
-

Q3.

1. Database Vs Table

- Database: The main file stored on the device (eg. posts.db)
- Table: A structure inside the database that stores data in rows and columns (eg. posts table)

Example / syntax:

// open or create database.

final db = await openDatabase (

'posts.db',

version: 1,

onCreate: (db, version) async {

// create table inside database

await db.execute (""

CREATE TABLE POSTS (

id INTEGER PRIMARY KEY AUTOINCREMENT,

title TEXT,

body TEXT,

)

"");

},

);

Thus the database contains the tables and each table stores multiple rows of structured data

6)11. CRUD operations

SQLite allows performing CRUD operations to manage data

• Create

```
await db.insert('posts', post.toMap());
```

• Read

```
List<Map<String, dynamic>> posts = await db.query  
('posts');
```

• update

```
await db.update(  
  'posts',  
  post.toMap(),  
  where: 'id = ?',  
  whereArgs: [post.id],  
);
```

• Delete

```
await db.delete(  
  'posts',  
  where: 'id = ?',  
  whereArgs: [post.id],  
);
```

iii. How flutter interacts with the database asynchronously.

→ SQLite operations take time, so flutter handles them using asynchronous programming.

→ The keywords `async` and `await` are used to perform database operations without blocking the UI.

→ Results are returned as a Future, which is handled using widgets like `FutureBuilder` or by using `setState()`.

Syntax example:

```
Future <List <Map <String, dynamic>>>
```

```
getposts () async {
```

```
  final db = await openDatabase ('posts.db');
```

```
  return await db.query ('posts');
```

```
}
```

- SQLite in flutter allows storing structured data locally using databases and tables. It supports CRUD functions and interact asynchronously with the UI using `Future`, `async/await`, and widgets like `FutureBuilder`.

Conclusion

This lab helped in understanding how Flutter applications manage local data using SQLite. It demonstrated how to perform CRUD operations without internet and how to update the UI dynamically.

It also showed the importance of handling database errors properly to ensure a reliable and efficient mobile application.

